

Основы систем мобильной связи (ПОМС)

Отчет по Практической работе №5

Студент: Любимов К. А.

Группа: ИА-331

1. Цель работы:

Получить представление о том, как осуществляется проверка на наличие ошибок в пакетах с данными в современных системах связи (Error detection) посредством использования циклического избыточного кода CRC (Cyclic Redundancy Check).

2. Задание для выполнения практической работы:

В рамках данной работы студенты должны научиться вычислять CRC - последовательности, а также на их основании детектировать ошибки.

3. Теоретические сведения:

Псевдослучайные двоичные последовательности

CRC — циклический избыточный код, иногда называемый также контрольным кодом или контрольной суммой. CRC – это добавочная порция избыточных бит, вычисляемых по заранее известному алгоритму на основе исходного передаваемого пакета данных (информационной битовой последовательности), которое передаётся вместе с самим пакетом по каналам связи (добавляется после информационных битов) и служит для контроля его безошибочной передачи.

Простыми словами, CRC – это остаток от двоичного деления оригинального пакета с данными на какое-то двоичное n -разрядное число (порождающий полином), и его длина будет равна $n-1$ бит. Рассмотрим пример, где имеется 7 бит данных: 100100 и 4-битный порождающий полином 1101. Требуется определить CRC. Для того, чтобы выполнить деление этих битовых последовательностей нужно в конце последовательности с данными добавить $n-1$ нулей, как показано ниже, где $n=4$, для нашего случая.

Делитель - 1 1 0 1 | 1 0 0 1 0 0 0 0 - Делимое (данные+ $n-1$ нулей).

Основной операцией, используемой при делении бинарных чисел, является исключающее ИЛИ (XOR). Ниже показана таблица истинности для данной операции.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Делитель принято записывать в виде полинома. Если считать, что каждый разряд делителя — это коэффициент полинома, то этот полином будет иметь вид:

$$x^{n-1} + x^{n-2} + \dots + x^2 + x^1 + x^0$$

Таким образом, делитель из примера выше можно записать в виде полинома как: $1 \cdot x^3 + 1 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0$, или сокращенно как: $x^3 + x^2 + 1 = 1101$

Полученный остаток от деления CRC добавляется на передающей стороне к исходным данным и уже эта битовая последовательность, преобразованная в радиосигнал, передается в канал связи: 1 0 0 1 0 0 0 0 1.

На приемной стороне для обнаружения ошибки (или ее отсутствия) с полученным пакетом осуществляется ровно такая же процедура – деление на порождающий CRC полином. Если полученный в результате данного деления остаток будет ненулевым, то фиксируется факт наличия ошибки.

4. Порядок выполнения работы:

1) Напишите программу на языке C/C++ для вычисления CRC для пакета данных длиной N бит ($N = 20 + \text{порядковый номер в журнале}$) и определения факта наличия ошибки при передаче пакета по каналу связи.

2) Порождающий полином G для делителя выберите в соответствии с вариантом. Номер варианта – порядковый номер в журнале группы.

Табл. 5.1 Варианты заданий.

№ варианта	Непрерывная периодическая функция
16	$G = x^7 + x^2 + x + 1$

3) Добавьте полученный остаток от деления на G к пакету исходными данными и на приемной стороне вычислите повторно остаток от деления пакета с данными+CRC на полином G. Определите есть ли ошибка в принятом пакете. Выведите в окно терминала полученное значение CRC и отчет об ошибках в принятом пакете.

4) Возьмите N, равное 250 битам. Прodelайте п.1-3

5) Сделайте цикл из $250 + \text{CRC length}$ итераций и в этом цикле по очереди искажайте по одному биту – с 0-го до $250 + \text{CRC} - 1$, проверьте в

соответствии с п.3 обнаружена ли ошибка на приемной стороне и выполните подсчет того сколько раз за этот цикл приемник обнаружил и не обнаружил ошибки. Результат выведите в окно терминала.

6) Составьте отчет. Отчет должен содержать титульный лист, содержание, цель и задачи работы, теоретические сведения, исходные данные, этапы выполнения работы, сопровождаемые скриншотами и графиками, демонстрирующими успешность выполнения, и промежуточными выводами, результирующими таблицами, ответы на контрольные вопросы, и заключение и ссылка в виде QR-кода на репозиторий с кодом (git).

5. Выполнения работы:

Я использовал язык C++ для реализации лабораторной работы. Формулы, который я использовал для выполнения задания, находятся в [методичке](#). Код лежит в репозитории GitHub, ссылка на который приложена в конце отчета.

6. Результаты:

Выводо кода на C++:

```
Передача бит, с пораздающим полиномом [0, 0, 1, 1]:
CRC код: 010
No Errors in data!

Передача бит, с пораздающим полиномом [1, 0, 0, 0, 0, 1, 1, 1]:
CRC код: 1110101
No Errors in data!

Передача бит, с пораздающим полиномом [0, 0, 1, 1], N = 250:
CRC код: 111
No Errors in data!

Передача бит, с пораздающим полиномом [1, 0, 0, 0, 0, 1, 1, 1], N = 250:
CRC код: 0000111
No Errors in data!

Если бит 0 искажился: ERROR
Если бит 1 искажился: ERROR
Если бит 2 искажился: ERROR
Если бит 3 искажился: ERROR
Если бит 4 искажился: ERROR
Если бит 5 искажился: ERROR
Если бит 6 искажился: ERROR
Если бит 7 искажился: ERROR
Если бит 8 искажился: ERROR
```

[illegible]

[illegible]

[illegible]

[illegible]

Если бит 217 исказился: ERROR
Если бит 218 исказился: ERROR
Если бит 219 исказился: ERROR
Если бит 220 исказился: ERROR
Если бит 221 исказился: ERROR
Если бит 222 исказился: ERROR
Если бит 223 исказился: ERROR
Если бит 224 исказился: ERROR
Если бит 225 исказился: ERROR
Если бит 226 исказился: ERROR
Если бит 227 исказился: ERROR
Если бит 228 исказился: ERROR
Если бит 229 исказился: ERROR
Если бит 230 исказился: ERROR
Если бит 231 исказился: ERROR
Если бит 232 исказился: ERROR
Если бит 233 исказился: ERROR
Если бит 234 исказился: ERROR
Если бит 235 исказился: ERROR
Если бит 236 исказился: ERROR
Если бит 237 исказился: ERROR
Если бит 238 исказился: ERROR
Если бит 239 исказился: ERROR
Если бит 240 исказился: ERROR
Если бит 241 исказился: ERROR
Если бит 242 исказился: ERROR
Если бит 243 исказился: ERROR
Если бит 244 исказился: ERROR
Если бит 245 исказился: ERROR
Если бит 246 исказился: ERROR
Если бит 247 исказился: ERROR
Если бит 248 исказился: ERROR
Если бит 249 исказился: OK
Если бит 250 исказился: ERROR
Если бит 251 исказился: ERROR
Если бит 252 исказился: ERROR
Если бит 253 исказился: ERROR
Если бит 254 исказился: ERROR
Если бит 255 исказился: ERROR
Если бит 256 исказился: ERROR

7. Контрольные вопросы:

1) Для чего в мобильных сетях используются CRC-проверки?

CRC используется для обнаружения ошибок при передаче данных по каналу связи.

CRC позволяет приёмнику:

- понять, исказился ли пакет
- принять решение:
 - отбросить пакет
 - запросить повторную передачу
 - или передать данные дальше

CRC не исправляет ошибки, а только сообщает, что данные стали недостоверными.

2) Что такое порождающий полином?

Порождающий полином — это фиксированное двоичное число, используемое как делитель при вычислении CRC.

Он:

- задаёт правило вычисления контрольной суммы
- определяет, какие ошибки будут обнаружены, а какие — нет

Каждый коэффициент полинома соответствует одному биту.

Например:

$$G = x^7 + x^2 + x + 1 == 10000111$$

3) Как вычислить CRC для пакета с данными

CRC вычисляется так:

1. Берётся исходная последовательность данных
2. В конец данных добавляются $n-1$ нулей, где n — длина порождающего полинома
3. Полученная последовательность делится по модулю 2 (через XOR) на порождающий полином
4. Остаток от деления длиной $n-1$ бит — это CRC
5. CRC приписывается в конец исходных данных и передаётся вместе с ними

На приёмной стороне:

- весь принятый пакет (данные + CRC) снова делится на тот же полином
- если остаток равен нулю — пакет принят без ошибок
- если остаток ненулевой — обнаружена ошибка

8. Вывод:

В ходе лабораторной работы был изучен принцип работы циклического избыточного кода CRC и реализован алгоритм вычисления контрольной суммы для двоичных пакетов данных. Было показано, что добавление CRC к пакету и последующая проверка на приёмной стороне позволяют обнаруживать большинство ошибок, возникающих при передаче по каналу связи. Эксперимент с поочерёдным искажением отдельных битов продемонстрировал, что выбранный порождающий полином эффективно детектирует ошибки, однако существуют отдельные позиции битов, при искажении которых ошибка не обнаруживается, что отражает ограниченные, но предсказуемые возможности CRC как метода контроля целостности данных.



Ссылка и QR на [GitHub](#)